

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Debugging & Optimization Task

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	Assessment:	Assessment Tool 2 – Debugging & Optimisation Task
Weightage:	35% of CIA	Total Marks:	100
CO2 Statement:	Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.		
Student Name:		Reg. No.:	
Section / Batch:		Date:	22/07/2026

Code of Conduct

I SRIVATSAN L(192521480) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: SRIVATSAN L

Instructions

- This test contains 20 Debugging & Optimisation Task questions. Total: 100 Marks (20*5).
- When a student answer using Scenario-Based, in evaluation the answer should be with following five requirements: Identify the problem type and constraints, Select appropriate data structure, Design the Solution, Optimization and efficiency, Clarity of representation.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
Search Data Structure, Stack	Binary Search, Stack Operations	1 - 5	25
Linked List Data Structure	List Operations	6 – 10	25
Queue Data Structures	Queue Operations	11 - 15	25
Queue Data Structures, Stack Notations	Queue Operations, Expression Evaluation	16 - 20	25
GRAND TOTAL:			100 Marks

SET 1

QUESTIONS

Q1

Binary Search
5 marks



Q2

Stack
5 marks



Q3

Stack
5 marks



Q4

Stack
5 marks



Q5

Stack
5 marks



Q6

Linked List
5 marks



Q7

Stack
5 marks



Q8

Linked List Data Structures
5 marks



Q1 Binary Search

5 marks

Identify the error in the following snippet.

```
int binarySearch(int arr[], int n, int key) {  
    int low = 0, high = n - 1;  
    while(low < high) {  
        int mid = (low + high) / 2;  
        if(arr[mid] == key)  
            return mid;  
        else if(arr[mid] < key)  
            low = mid;  
        else  
            high = mid;  
    }  
    return -1;  
}
```

 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 int binarySearch(int arr[], int n, int key){  
2     int low=0,high=n-1;  
3     while(low<=high){  
4         int mid=(low+high)/2;  
5         if(arr[mid]==key)  
6             return mid;
```

Test Input

stdin input (optional)

Output

Q2 Stack

5 marks

Find the error in the stack deletion operation. Justify it.

```
int pop() {  
    if(top == 0) {  
        printf("Underflow");  
        return -1;  
    }  
    return stack[top--];  
}
```

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 int pop() {  
2     if (top== -1) {  
3         printf("Underflow");  
4         return -1;  
5     }  
6     return stack[top--];  
7 }
```

Test Input

```
10 20 30  
2
```

Output

```
Compilation Error:  
C:\Windows\TEMP\sandbox_192521480  
RB_6a70529eeafe38.12804198\main.c  
: In function 'pop':  
C:\Windows\TEMP\sandbox_192521480  
RB_6a70529eeafe38.12804198\main.c
```

Q3 Stack

5 marks

Check the snippet and identify the logical error.

```
int stack[5];  
int top;  
void init() {  
    top = 0;  
}
```

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 int stack[5];  
2 int top;  
3 void init() {  
4     top=-1;  
5 }
```

Test Input

stdin input (optional)

Output

```
Compilation Error:  
C:/CCPPComp/CCPP2/bin/./lib/gcc/  
x86_64-w64-  
mingw32/10.3.0/../../../../x86_64-  
w64-mingw32/bin/ld.exe:  
C:/CCPPComp/CCPP2/bin/./lib/gcc/
```

🕒 Pending manual review

Q4 Stack

5 marks

Identify the errors in the following snippet.

```
if(ch == ')') {  
while(stack[top] != ')') {  
postfix[j++] = pop0;  
}  
pop0;  
}
```

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 if(ch==' '){  
2     while(stack[top]!=' '){  
3         postfix[j++]=pop();  
4     }  
5     pop();  
6 }
```

Test Input

stdin input (optional)

Output

Q5 Stack

In balancing Bracket, you identify the error.

```
if((ch == ']' && stack[top] == '[') ||  
(ch == '}' && stack[top] == '{')) {  
pop0;  
} else {  
return 0;  
}
```

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the

Your Fixed Code

```
1 if ((ch == '[' && stack[top] == '[') ||  
2     (ch == ']' && stack[top] == '[') ||  
3     (ch == '}' && stack[top] == '{')) {  
4     pop();  
5 } else {  
6     return 0;  
7 }
```

Q6 Linked List

5 marks

What is the issue?

```
temp = head
```

```
while temp.next != NULL:
```

```
print(temp.data)
```

```
temp = temp.next
```

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 temp = head;
2
3 while (temp != NULL) {
4     printf("%d ", temp->data);
5     temp = temp->next;
6 }
```

Test Input

stdin input (optional)

Output

🕒 Pending manual review

Q7 Stack

Point out the error.

```
POP(stack):
```

```
if top == 0:
```

```
print("Underflow")
```

```
else:
```

```
return stack[top--]
```

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 POP(stack):
2     if top == -1:
3         print("Underflow")
4     else:
5         return stack[top--]
```

Test Input

stdin input (optional)

Output

Q8 Linked List Data Structures

Identify the errors in the following code

```
while current != NULL:
current.next = prev
prev = current
current = current.next
```

🔧 **Debugging Task:** Find and fix the bugs in the code shown

Your Fixed Code

```
1 while (current != NULL) {
2     temp = current->next;
3     current->next = prev;
4     prev = current;
5     current = temp;
6 }
```

Q9 Linked List

What is the error in the below code?

```
new.next = head
head = new
```

🔧 **Debugging Task:** Find and fix the bugs in the code shown

Your Fixed Code

```
1 new->next = head;
2 head = new;
```

Q10 Queue

5 marks

What is missing in the below pseudocode?

```
DEQUEUE(Q):
if front == -1:
print("Underflow")
else:
return Q[front++]
```

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 DEQUEUE(Q) :
2     if front == -1:
3         print("Underflow")
4     else:
5         item = Q[front]
6         front = front + 1
7
8         if front > rear:
9             front = -1
10            rear = -1
11
12            return item
```

Test Input

stdin input (optional)

Output

SET- 2

Find the bug in the evaluation of a Postfix expression.

```
while (expression[i] != '\0') {  
    if (isdigit(expression[i])) push(expression[i] - '0');  
    else {  
        int op1 = pop();  
        int op2 = pop();  
        // perform operation: result = op1 + op2;  
        push(result);  
    }  
}
```

✳ **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor.

Your Fixed Code

```
1 while (expression[i] != '\0') {  
2     if (isdigit(expression[i]))  
3         push(expression[i] - '0');  
4     else {  
5         int op2 = pop();  
6         int op1 = pop();  
7  
8         // perform operation  
9         result = op1 + op2;  
10  
11         push(result);  
12     }  
13 }
```

Te

O

Q2 Circular Queue

Identify the bug in this Circular Queue dequeue logic.

```
int dequeue() {  
    if (front == -1) return -1;  
    int data = queue[front];  
    front = (front + 1) % MAX;  
    return data;  
}
```

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor.

Your Fixed Code

```
1 int dequeue() {  
2     if (front == -1)  
3         return -1;  
4  
5     int data = queue[front];  
6  
7     if (front == rear) {  
8         front = -1;  
9         rear = -1;  
10    } else {  
11        front = (front + 1) % MAX;  
12    }  
13  
14    return data;  
15 }
```

Q3 Binary Search

Identify the bug in this Recursive Binary Search.

```
int binarySearch(int arr[], int l, int r, int x) {  
    int mid = l + (r - l) / 2;  
    if (arr[mid] == x) return mid;  
    if (arr[mid] > x) return binarySearch(arr, l, mid, x);  
    return binarySearch(arr, mid, r, x);  
}
```

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor.

Your Fixed Code

```
1 int binarySearch(int arr[], int l, int r, int x) {  
2     if (l > r)  
3         return -1;  
4  
5     int mid = l + (r - l) / 2;  
6  
7     if (arr[mid] == x)  
8         return mid;  
9  
10    if (arr[mid] > x)  
11        return binarySearch(arr, l, mid - 1, x);  
12  
13    return binarySearch(arr, mid + 1, r, x);  
14 }
```

Q4 Queue

Find the error in this Dequeue insertFront operation.

```
void insertFront(int data) {  
    if (front == 0) front = MAX - 1;  
    else front = front - 1;  
    deque[front] = data;  
}
```

🔗 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the box below.

Your Fixed Code

```
1 void insertFront(int data) {  
2     if (front == 0)  
3         front = MAX - 1;  
4     else  
5         front = front - 1;  
6  
7     deque[front] = data;  
8 }
```

Q5 Queue

Identify the error in the Queue enqueue logic.

```
void enqueue(int data) {  
    if (rear == MAX) return;  
    queue[rear++] = data;  
}
```

🔗 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the box below.

Your Fixed Code

```
1 void enqueue(int data) {  
2     if (rear == MAX - 1)  
3         return;  
4  
5     queue[rear++] = data;  
6 }
```

Q6 Doubly Linked List

Identify the error in this Doubly Linked List node deletion.

```
void deleteNode(Node* del) {  
    if (del->prev != NULL)  
        del->prev->next = del->next;  
    if (del->next != NULL)  
        del->next->prev = del->prev;  
}
```

✂ **Debugging Task:** Find and fix the bugs in the code shown above. Write y

Your Fixed Code

```
1 void deleteNode(Node* del) {  
2     if (del->prev != NULL)  
3         del->prev->next = del->next;  
4     else  
5         head = del->next;  
6  
7     if (del->next != NULL)  
8         del->next->prev = del->prev;  
9 }
```

Q7 Stack

Identify the issue in the below pseudocode.

POP():

if top == 0 → Underflow

✂ **Debugging Task:** Find and fix the bugs in the code shown above. W

Your Fixed Code

```
1 POP() {  
2     if (top == -1)  
3         printf("Underflow");  
4     else {  
5         item = stack[top];  
6         top--;  
7         return item;  
8     }  
9 }
```

Q8 Stack

Point out the error.

if top == MAX:

Overflow

🔧 **Debugging Task:** Find and fix the bugs:

Your Fixed Code

```
1 PUSH(item) {
2     if (top == MAX - 1)
3         printf("Overflow");
4     else {
5         top++;
6         stack[top] = item;
7     }
8 }
```

Q9 Circular Queue

Identify the issue in the following snippet for Circular Queue in Traffic Signal.

rear = (rear + 1) % MAX;

queue[rear] = vehicle;

if(rear == front){

printf("Queue Full");

}

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your code

Your Fixed Code

```
1 if ((rear + 1) % MAX == front) {
2     printf("Queue Full");
3 }
4 else {
5     rear = (rear + 1) % MAX;
6     queue[rear] = vehicle;
7 }
```

Q10 Stack

Point out the error?

PEEK(stack):

return stack[top+1]

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code

Your Fixed Code

```
1 PEEK(stack) {
2     if (top == -1)
3         printf("Underflow");
4     else
5         return stack[top];
6 }
```

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Debugging Accuracy	20	All errors corrected; code runs flawlessly	Most errors corrected; minor issues remain	Partial correction; code runs with issues	Code fails or major errors persist
Error Identification	30	Accurately identifies all logical, syntax, and edge-case errors	Identifies most major errors	Identifies some errors but misses key issues	Struggles to identify errors
Code Quality & Readability	20	Clean, modular, well-documented, follows best practices	Mostly clean and readable	Some structure but inconsistent	Poorly written, hard to understand
Explanation & Justification	20	Clear, logical, and well-justified reasoning with complexity analysis	Good explanation with some justification	Limited explanation; lacks depth	No or incorrect explanation
Testing & Validation	10	Comprehensive test cases including edge cases	Adequate test cases	Minimal testing	No testing provided

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____